

Glosario integrador del módulo 2

PF1822 · Especialización en Desarrollo con IA

Módulo 2 · MA04576 · Introducción a modelos generativos y consumo por API

TÉRMINOS

61

SE USA

Durante todo el módulo, y en Moodle como actividad Glosario

ORGANIZACIÓN

Por contenido del plan, con índice alfabético

CONTENIDOS DEL PLAN

Los 39 temas de los 4 contenidos del módulo tienen al menos un término

Este glosario reúne en un solo lugar el vocabulario técnico del módulo «Introducción a modelos generativos y consumo por API». Integra y amplía los glosarios de las cuatro lecturas: cada término tiene una definición precisa, un ejemplo tomado de Nube Sur, la empresa ficticia de software cuya mesa de ayuda acompaña todo el módulo, y los términos relacionados con los que conviene leerlo.

Está organizado por los cuatro contenidos del plan formativo, en el mismo orden en que avanza el módulo:

- 1. Modelos generativos y ecosistema tecnológico:** qué es la IA generativa, qué tipos de modelos existen, para qué se usan y cómo se arma una aplicación que los integra.
- 2. Integración y consumo de APIs:** cómo se consume un modelo por API REST desde Python, con autenticación segura, manejo de errores, docstrings y pruebas.
- 3. Diseño de prompts para APIs generativas:** cómo se escribe, se parametriza y se mejora un prompt, y qué límites tiene el modelo.
- 4. Pipeline de preprocesamiento y evaluación básica de texto:** cómo se limpia, normaliza y tokeniza un texto, y cómo se miden y registran los resultados para decidir ajustes.

Dentro de cada contenido, los términos van en orden alfabético, y la última columna indica el tema del plan formativo que cada término ayuda a comprender; todos los temas del módulo tienen al menos un término. Al final, un índice alfabético reúne todos los términos con el número de su contenido. Los términos que las herramientas y la documentación usan en inglés (embedding, endpoint, few-shot, temperature, top_p) se dejan en inglés, tal como los vas a encontrar al programar.

Cómo usarlo:

- **Durante las lecturas.** Cuando una lectura use un término de otro contenido, búscalo aquí. Los *Relacionados* te llevan de un contenido a otro: de Embedding a Similitud coseno, o de Token a Max_tokens.

- **En los ABP y ABPRO.** Úsalo para nombrar con precisión lo que haces: al justificar una arquitectura, al escribir las docstrings de tu cliente de API, al registrar una iteración de prompt o al definir un protocolo de evaluación.
- **En la actividad final integradora.** El asistente de preguntas frecuentes de Nube Sur usa términos de los cuatro contenidos a la vez, y muchos ejemplos de este glosario salen de ese caso. Revísalos antes de empezar y úsalos al escribir tu informe.
- **En Moodle.** El mismo glosario está en el curso como actividad Glosario: puedes buscar un término, recorrerlo por letra o por categoría (una por cada contenido del plan). El PDF y la actividad tienen los mismos términos, definiciones y ejemplos; los términos relacionados aparecen en el PDF.

Contenido 1 · Modelos generativos y ecosistema tecnológico

Término	Definición	Contenido del plan
Embedding	Representación de un texto como un vector de números de largo fijo, calculada por un modelo de embeddings (en general, un transformer codificador), en la que los textos de significado parecido quedan cerca. Se obtiene del endpoint de embeddings y se compara con la similitud coseno. <i>Ejemplo:</i> «olvidé la clave y no puedo entrar» y «¿Cómo recupero mi contraseña?» no comparten palabras clave, pero sus embeddings quedan cerca, y así el asistente elige la pregunta frecuente F1. <i>Relacionados:</i> Vector store, Similitud coseno, Endpoint, Transformer	Componentes: cliente, servidor API, modelo de IA, módulo de preprocesamiento y base de datos/vector store.
Hugging Face Transformers	Biblioteca de Python de Hugging Face para descargar y ejecutar modelos abiertos del Hub (generación, resumen, clasificación, embeddings) sobre PyTorch: desde la versión 5 funciona solo con PyTorch, y el soporte de TensorFlow y JAX quedó en la 4.x. PyTorch y TensorFlow son los dos frameworks de aprendizaje profundo más usados para entrenar y ejecutar modelos. Es parte del ecosistema tecnológico de la IA junto a las APIs de proveedores como OpenAI y a bibliotecas como LangChain y Diffusers. <i>Ejemplo:</i> Si Nube Sur quisiera que las preguntas de sus clientes no salieran de la empresa, podría calcular los embeddings con un modelo abierto ejecutado con Transformers en su servidor; en esta versión le basta la API. <i>Relacionados:</i> LangChain, Transformer, Referencia de la API	Ecosistema tecnológico de la IA. Flujo de datos end-to-end: tecnologías y librerías: OpenAI API, Hugging Face Transformers, TensorFlow, PyTorch, LangChain y Diffusers.

Término	Definición	Contenido del plan
IA basada en reglas	Enfoque tradicional en que el sistema decide con condiciones escritas por personas («si el texto contiene X, hacer Y»). Es predecible, barato y fácil de auditar, pero solo resuelve los casos que alguien anticipó y no produce contenido nuevo. <i>Ejemplo:</i> Asignar a Facturación todo ticket que contenga «factura» es una buena regla, pero un buscador por palabras clave no reconoce que «olvidé la clave y no puedo entrar» pregunta lo mismo que F1, «¿Cómo recupero mi contraseña?». <i>Relacionados:</i> IA generativa, Embedding	Diferencias entre IA generativa y IA tradicional basada en reglas.
IA generativa	Rama de la inteligencia artificial que produce contenido nuevo (texto, código, imágenes, audio o video) a partir de patrones aprendidos de grandes cantidades de datos; los modelos multimodales combinan más de uno de esos tipos. Sirve para redactar, resumir, responder preguntas, generar código y pruebas, pero su salida es probable, no verificada, y siempre hay que validarla. <i>Ejemplo:</i> Nube Sur usa IA generativa para escribir una oración por ticket y para redactar respuestas a partir de sus preguntas frecuentes, dos tareas que ninguna regla fija puede anticipar. <i>Relacionados:</i> IA basada en reglas, Modelo de lenguaje, Alucinación	Conceptos generales de IA generativa: qué es la IA generativa y para qué sirve. Tipos de IA generativa: texto, imagen, audio y video.
LangChain	Framework para orquestar aplicaciones con modelos de lenguaje: encadena prompts, llamadas a modelos, recuperación de documentos, herramientas y agentes, con una interfaz común para distintos proveedores. <i>Ejemplo:</i> El asistente de Nube Sur no lo necesita en esta versión, porque su flujo es corto y cabe en un cliente de API propio; se vuelve útil cuando el flujo suma pasos, fuentes de datos y herramientas. <i>Relacionados:</i> Hugging Face Transformers, Pipeline, Modularización	Ecosistema tecnológico de la IA. Flujo de datos end-to-end: tecnologías y librerías: OpenAI API, Hugging Face Transformers, TensorFlow, PyTorch, LangChain y Diffusers.
Modelo autorregresivo	Modelo que genera la salida un token a la vez: predice el siguiente a partir de la entrada y de todo lo que ya generó, lo agrega y repite hasta terminar o llegar al límite de tokens. Así generan texto y código los modelos de lenguaje actuales. <i>Ejemplo:</i> La respuesta del asistente de Nube Sur se escribe token a token, y por eso un <code>max_tokens</code> demasiado bajo puede cortarla a mitad de una oración. <i>Relacionados:</i> Modelo de lenguaje, Transformer, Max_tokens	Tipos de modelos: autorregresivos, de difusión y basados en Transformers.

Término	Definición	Contenido del plan
Modelo de difusión	Modelo que genera imágenes, audio o video partiendo de ruido aleatorio y refinándolo en muchos pasos hasta obtener un resultado que corresponde a la descripción pedida. Es el tipo de modelo más usado en la IA generativa de imagen y video, y también se aplica a audio; en Python se usan, entre otras, con la biblioteca Diffusers de Hugging Face. <i>Ejemplo:</i> Nube Sur lo descarta para el asistente de preguntas frecuentes, que solo produce texto, pero le serviría para ilustrar los artículos de su centro de ayuda. <i>Relacionados:</i> IA generativa, Modelo autorregresivo	Tipos de modelos: autorregresivos, de difusión y basados en Transformers. Tipos de IA generativa: texto, imagen, audio y video.
Modelo de lenguaje	Modelo de IA generativa de texto que, entrenado con enormes corpus, predice el token siguiente de una secuencia; los modelos grandes (LLM) así redactan, resumen, responden preguntas, siguen instrucciones y escriben código. Es el tipo de modelo que este módulo consume por API. <i>Ejemplo:</i> El modelo que Nube Sur configura en la variable <code>MODELO_CHAT</code> es un modelo de lenguaje: recibe la consulta y la respuesta publicada elegida, y redacta la respuesta del asistente. <i>Relacionados:</i> Modelo autorregresivo, Transformer, Prompt, Token	Tipos de IA generativa: texto, imagen, audio y video. Tipos de modelos: autorregresivos, de difusión y basados en Transformers.
Pipeline	Arquitectura de etapas encadenadas en que la salida de cada etapa es la entrada de la siguiente. Cada etapa se prueba y se cambia por separado, y el conjunto puede procesar muchos elementos en lote, sin que nadie espere en pantalla. <i>Ejemplo:</i> Los embeddings de las preguntas frecuentes F1 a F8 se calculan una sola vez en un pipeline (limpiar, normalizar, obtener embeddings, guardar en el vector store) y no en cada consulta, lo que ahorra llamadas a la API. <i>Relacionados:</i> Solicitud–respuesta, Pipeline de preprocesamiento, Vector store	Arquitecturas: request–response y pipelines.
QA (preguntas y respuestas)	Caso de uso en que el modelo responde preguntas formuladas en lenguaje natural. En una aplicación confiable se le entrega el texto de referencia (documentación, preguntas frecuentes) y se le exige responder solo con él, sin inventar. <i>Ejemplo:</i> El asistente de Nube Sur es un caso de QA: busca la pregunta frecuente más parecida a la consulta y responde solo con su respuesta publicada, o deriva con un ejecutivo. <i>Relacionados:</i> Summarization, Restricción, Alucinación, Embedding	Casos de uso: generación de texto, generación de código, síntesis de imágenes, QA, summarization y generación de tests.

Término	Definición	Contenido del plan
Servidor API	Componente intermedio que recibe las solicitudes del cliente (la aplicación que usa la persona), orquesta el flujo de datos de extremo a extremo por el preprocesamiento, el modelo de IA y la base de datos o vector store, y devuelve el resultado. Es el único componente que guarda la clave de la API. <i>Ejemplo:</i> El flujo end-to-end del asistente es: el centro de ayuda envía la pregunta al servidor, que la limpia, obtiene su embedding, la compara en el vector store, llama al modelo y devuelve la respuesta, sin que el navegador vea nunca la clave. <i>Relacionados:</i> Vector store, Variable de entorno, Solicitud–respuesta, Bearer	Componentes: cliente, servidor API, modelo de IA, módulo de preprocesamiento y base de datos/vector store. Flujo de datos end-to-end: tecnologías y librerías: OpenAI API, Hugging Face Transformers, TensorFlow, PyTorch, LangChain y Diffusers.
Solicitud–respuesta	Arquitectura (<i>request–response</i>) en que la aplicación envía una solicitud al modelo, espera y muestra la respuesta; cada solicitud es independiente de las anteriores. Es simple y sirve para respuestas inmediatas, pero la persona espera mientras el modelo genera. <i>Ejemplo:</i> Cuando alguien escribe una pregunta en el centro de ayuda, el asistente de Nube Sur responde con una solicitud–respuesta: una consulta, una respuesta en pocos segundos. <i>Relacionados:</i> Pipeline, Servidor API, API REST	Arquitecturas: request–response y pipelines.
Summarization	Caso de uso en que el modelo condensa un texto largo en uno más corto que conserva lo esencial, sin agregar datos que no estaban en el original. Es una tarea fiel a la fuente: el modelo reformula con sus palabras lo que ya está (resumen abstractivo), pero no crea información nueva; por eso se hace con temperature baja y se evalúa contra un resumen de referencia. <i>Ejemplo:</i> El resumidor de Nube Sur convierte el ticket T1, con HTML y firma, en «El usuario no puede iniciar sesión en la app de facturación tras cambiar su contraseña». <i>Relacionados:</i> QA (preguntas y respuestas), ROUGE, Temperature	Casos de uso: generación de texto, generación de código, síntesis de imágenes, QA, summarization y generación de tests.
Transformer	Arquitectura de red neuronal basada en el mecanismo de atención, que relaciona cada parte de la entrada con todas las demás; en ella se apoyan casi todos los modelos de lenguaje actuales. Tiene variantes codificadoras (convierten un texto en vectores), decodificadoras (generan texto de forma autorregresiva) y codificador–decodificador (transforman una entrada en otra salida, como en una traducción). <i>Ejemplo:</i> En el asistente de Nube Sur trabajan dos transformers: uno codificador calcula los embeddings de las preguntas y uno decodificador redacta la respuesta. <i>Relacionados:</i> Modelo autorregresivo, Embedding, Modelo de lenguaje	Tipos de modelos: autorregresivos, de difusión y basados en Transformers.

Término	Definición	Contenido del plan
Vector store	Base de datos que guarda embeddings junto al texto al que corresponden y permite buscar los más parecidos a un vector dado. En una versión mínima puede ser una lista en memoria o un archivo; a mayor escala, un servicio especializado. <i>Ejemplo:</i> El vector store del asistente guarda los embeddings de F1 a F8, y cada consulta se compara contra ellos sin volver a pedirlos a la API. <i>Relacionados:</i> Embedding, Similitud coseno, Servidor API	Componentes: cliente, servidor API, modelo de IA, módulo de preprocesamiento y base de datos/vector store.

Contenido 2 · Integración y consumo de APIs

Término	Definición	Contenido del plan
API REST	Interfaz que expone recursos en URL y se usa con métodos HTTP: cada solicitud es independiente, lleva su autenticación y recibe un código de estado y un cuerpo, casi siempre en JSON. GET lee un recurso sin enviar cuerpo; POST envía datos en el cuerpo y pide un resultado. <i>Ejemplo:</i> Al arrancar, el servidor de Nube Sur hace un GET a <code>/v1/models</code> para comprobar la clave y el modelo configurado, y después cada consulta del asistente es un POST con cuerpo JSON. <i>Relacionados:</i> Endpoint, Código de estado, Parsing de respuesta	Principios REST y métodos HTTP: GET y POST.
API simulada	Sustituto de la API real que responde lo que la prueba define, sin internet, sin clave y sin costo, para que las pruebas den siempre el mismo resultado. Con httpx se arma con <code>httpx.MockTransport</code>; con requests, con <code>unittest.mock</code>. <i>Ejemplo:</i> Para probar la cuota agotada, la API simulada responde 429 y la prueba verifica que <code>ClienteIA</code> lanza <code>ErrorAPI</code> con ese código, sin reintentar. <i>Relacionados:</i> Prueba unitaria, httpx, Código de estado	Pruebas unitarias simples para funciones de consumo de APIs.
Bearer	Esquema de autenticación en que la clave se envía en el encabezado <code>Authorization</code> de cada solicitud, precedida de la palabra <code>Bearer</code> y un espacio. Lo usan la API de OpenAI, con la clave de API, y Hugging Face, con un token de acceso. <i>Ejemplo:</i> <code>ClienteIA</code>, que corre en el servidor de Nube Sur, agrega una sola vez el encabezado <code>Authorization: Bearer</code> con la clave leída de <code>OPENAI_API_KEY</code>, y una prueba unitaria verifica que viaje en cada solicitud. <i>Relacionados:</i> Variable de entorno, Código de estado, Prueba unitaria	Gestión de autenticación y variables de entorno.

Término	Definición	Contenido del plan
Chat completions	Endpoint de generación (POST <code>/v1/chat/completions</code>) que recibe un cuerpo JSON con <code>model</code>, la lista <code>messages</code> con sus roles (<code>system</code>, <code>user</code>, <code>assistant</code>) y parámetros como <code>temperature</code> y <code>max_tokens</code>, y devuelve el texto en <code>choices[0].message.content</code>. OpenAI ofrece además la API Responses; aquí se usa chat completions porque muchos proveedores, incluido Hugging Face, replican su formato. <i>Ejemplo:</i> El método <code>generar(mensajes, temperature, max_tokens)</code> de <code>ClienteIA</code>, en el servidor de Nube Sur, arma ese cuerpo con el mensaje de sistema, los ejemplos y la consulta recibida. <i>Relacionados:</i> Endpoint, Mensaje de sistema, Parsing de respuesta, Temperature	Endpoints de generación y embeddings. Ejemplos de uso: parámetros, cuerpos JSON y parsing de respuesta.
Código de estado	Número de tres cifras con que una respuesta HTTP informa el resultado: 2xx éxito; 4xx error de quien hace la solicitud (400 cuerpo inválido, 401 clave ausente o inválida, 404 recurso o modelo inexistente, 429 demasiadas solicitudes o cuota agotada); 5xx falla del servicio. Se reintentará, con esperas crecientes y respetando el encabezado <code>Retry-After</code> si viene, un 429 por límite de solicitudes, un 5xx o un tiempo de espera agotado, que es un error sin código; un 429 por cuota o saldo agotado (<code>insufficient_quota</code> en OpenAI), igual que un 400, 401 o 404, no se arregla reintentando. <i>Ejemplo:</i> Con una clave revocada, <code>ClienteIA</code> recibe 401 y lanza <code>ErrorAPI</code> con «HTTP 401», en vez de caerse con un <code>KeyError</code> al buscar <code>choices</code>. <i>Relacionados:</i> API REST, Parsing de respuesta, httpx, API simulada	Principios REST y métodos HTTP: GET y POST. Manejo básico de errores y respuestas del servicio.
Docstring	Texto de documentación entre comillas triples al comienzo de un módulo, clase o función de Python, que explica qué hace, qué recibe (Args), qué devuelve (Returns) y qué errores lanza (Raises). Python lo guarda con el código: <code>help()</code> y los editores lo muestran al usarlo. <i>Ejemplo:</i> La docstring de <code>embeddings()</code> avisa que devuelve un vector por texto en el mismo orden de entrada y que lanza <code>ErrorAPI</code> si el servicio responde con error. <i>Relacionados:</i> Modularización, Prueba unitaria	Documentación con docstrings.
Endpoint	URL de una API que ofrece una operación concreta con un método definido. En este módulo importan dos: el de generación (POST <code>/v1/chat/completions</code>) y el de embeddings (POST <code>/v1/embeddings</code>), que recibe en <code>input</code> un texto o una lista de textos y devuelve en <code>data</code> un vector por texto, con su posición en <code>index</code>. <i>Ejemplo:</i> <code>ClienteIA</code>, la clase con que el servidor de Nube Sur llama a la API, envía las ocho preguntas frecuentes en una sola llamada al endpoint de embeddings y ordena los vectores por <code>index</code> antes de guardarlos. <i>Relacionados:</i> Chat completions, Embedding, API REST, Referencia de la API	Endpoints de generación y embeddings.

Término	Definición	Contenido del plan
httpx	Biblioteca de Python para solicitudes HTTP, con una interfaz casi igual a la de requests, la otra biblioteca del módulo. A diferencia de requests, que espera sin límite si no se le indica <code>timeout</code>, httpx aplica un tiempo de espera de 5 segundos por defecto, ofrece un cliente asíncrono y permite simular la API en pruebas con <code>httpx.MockTransport</code>. Con cualquiera de las dos, cada solicitud debe llevar un tiempo de espera definido. <i>Ejemplo:</i> <code>ClienteIA</code> crea una vez <code>httpx.Client(base_url=..., timeout=30.0)</code> y reutiliza la conexión, la URL base y el encabezado de autenticación en todas sus llamadas; con requests, el equivalente es una <code>requests.Session()</code> y <code>timeout=30</code> en cada llamada. <i>Relacionados:</i> API simulada, Código de estado, Modularización	Librerías: requests y httpx.
Modularización	Reunir en una pieza de código (una función, una clase o un paquete) todo lo relativo a la API: URL base, autenticación, tiempo de espera, revisión de errores y cada operación. El resto de la aplicación llama a métodos con nombres claros y no sabe nada de HTTP, así que un cambio de proveedor o de formato toca un solo lugar. <i>Ejemplo:</i> Con el paquete <code>nubesur_ia</code>, el resto del código llama a <code>generar()</code> o <code>embeddings()</code>, y pasar de OpenAI a Hugging Face es cambiar <code>PROVEEDOR_IA</code>, no reescribir llamadas. <i>Relacionados:</i> Docstring, Prueba unitaria, Variable de entorno	Modularización de funciones o clases para consumo de APIs.
Parsing de respuesta	Leer la respuesta de la API y extraer lo que se necesita: primero se revisa el código de estado, después se convierte el cuerpo con <code>respuesta.json()</code> y se navega hasta los campos, como <code>choices[0].message.content</code>, <code>choices[0].finish_reason</code> y, en el primer nivel, <code>usage</code>, que trae los tokens de entrada, de salida y el total. <i>Ejemplo:</i> Por cada consulta, el asistente de Nube Sur muestra el texto generado y guarda los valores de <code>usage</code> en una fila de <code>consumo.csv</code>, para saber cuántos tokens gasta. <i>Relacionados:</i> Chat completions, Código de estado, Token, Salida estructurada	Ejemplos de uso: parámetros, cuerpos JSON y parsing de respuesta.
Prueba unitaria	Prueba automática que verifica una pieza de código de forma aislada y repetible; en un cliente de API comprueba que la solicitud va al endpoint y con el método correctos, que lleva la autenticación, que la respuesta se lee bien y que los errores se informan con su código. Se escribe con pytest y corre sin conexión, con la API simulada. <i>Ejemplo:</i> <code>test_embeddings_respeta_el_orden_de_entrada</code> entrega vectores desordenados por <code>index</code> y comprueba que <code>ClienteIA</code> los devuelve en el orden de los textos. <i>Relacionados:</i> API simulada, Docstring, Código de estado	Pruebas unitarias simples para funciones de consumo de APIs.

Término	Definición	Contenido del plan
Referencia de la API	Parte de la documentación oficial que describe cada endpoint: URL y método, autenticación, parámetros del cuerpo, forma de la respuesta y errores, con ejemplos. En OpenAI (platform.openai.com/docs) convive con las guías y las páginas de modelos y límites; en Hugging Face (huggingface.co/docs), la Inference API hoy se documenta como Inference Providers y se organiza por tarea, y cada modelo del Hub tiene su ficha (<i>model card</i>). <i>Ejemplo:</i> Antes de programar <code>embeddings()</code>, el README de Nube Sur anota, con el enlace a la página de cada proveedor, la URL, el método, la autenticación, los campos obligatorios y dónde viene el vector. <i>Relacionados:</i> Endpoint, Chat completions, Hugging Face Transformers	Documentación oficial: enlaces y organización de los manuales de OpenAI y Hugging Face inference API.
Variable de entorno	Valor con nombre que el sistema operativo o la plataforma entrega al programa al ejecutarlo; se usa para mantener claves y configuración fuera del código. En desarrollo se define en la terminal o en un archivo <code>.env</code> listado en <code>.gitignore</code>; Python no lee ese archivo por sí solo, así que el programa lo carga al iniciar, por ejemplo con <code>load_dotenv()</code> de <code>python-dotenv</code>. Un <code>.env.example</code> sin valores sirve de guía. <i>Ejemplo:</i> El paquete <code>nubesur_ia</code>, en el servidor de Nube Sur, lee <code>OPENAI_API_KEY</code> o <code>HF_TOKEN</code> según el valor de <code>PROVEEDOR_IA</code>, y en el repositorio solo queda <code>.env.example</code> con los nombres, nunca con los valores. <i>Relacionados:</i> Bearer, Servidor API, Modularización	Gestión de autenticación y variables de entorno.

Contenido 3 · Diseño de prompts para APIs generativas

Término	Definición	Contenido del plan
Alucinación	Respuesta que suena correcta pero contiene información falsa o que no estaba en la entrada; ocurre porque el modelo genera texto probable, no verificado. Se reduce con restricciones explícitas, temperature baja y validación posterior, pero no desaparece, así que lo generado se valida. <i>Ejemplo:</i> Si ante «¿cuánto cuesta el plan anual?» el asistente contesta con un precio, alucina: ninguna pregunta frecuente lo dice y debía derivar. <i>Relacionados:</i> Restricción, Prompt de validación, Temperature, Falta de control	Limitaciones: alucinaciones, falta de control y sensibilidad al wording.

Término	Definición	Contenido del plan
Falta de control	Limitación por la que no hay garantía de que el modelo cumpla todas las instrucciones siempre: la misma entrada puede dar salidas distintas, un formato pedido puede no respetarse y una regla puede ignorarse en un caso raro. Por eso el código que usa la respuesta la valida y tiene un camino seguro si la validación falla. <i>Ejemplo:</i> Aunque el prompt pide solo JSON, a veces el asistente antepone una frase o no usa la frase exacta de derivación; por eso el servidor de Nube Sur valida la salida con <code>json.loads</code> y deriva si falla. <i>Relacionados:</i> Alucinación, Salida estructurada, Sensibilidad al wording	Limitaciones: alucinaciones, falta de control y sensibilidad al wording.
Few-shot	Estrategia de prompting que agrega a las instrucciones de dos a cinco ejemplos resueltos de entrada y salida, que en la API de chat se escriben como turnos previos <code>user</code> y <code>assistant</code>. Fija el formato y el trato de los casos difíciles, pero suma tokens en cada llamada, el modelo tiende a imitar los ejemplos y estos deben ser distintos de los casos con que se evalúa. <i>Ejemplo:</i> Los dos ejemplos del prompt del asistente muestran una consulta respondida con su pregunta frecuente y otra derivada con «Te derivo con un ejecutivo», y ninguno repite las preguntas de prueba. <i>Relacionados:</i> Zero-shot, Mensaje de sistema, Iteración de prompts	Tipos de prompting: zero-shot, few-shot e instructional prompting.
Instructional prompting	Estrategia de prompting (prompting instruccional) que descompone la tarea en pasos numerados que el modelo debe seguir en orden. Sirve cuando la tarea tiene varias partes y se combina con zero-shot o few-shot. <i>Ejemplo:</i> Un prompt instruccional del asistente pide leer la consulta, decidir si la respuesta publicada la contesta, derivar si no la contesta y solo entonces devolver el JSON pedido. <i>Relacionados:</i> Zero-shot, Few-shot, Salida estructurada	Tipos de prompting: zero-shot, few-shot e instructional prompting.
Iteración de prompts	Mejorar un prompt con evidencia: probar, observar, cambiar una sola cosa y volver a probar con los mismos casos. Cada versión se registra con qué se cambió, por qué y qué efecto tuvo en claridad, estructura o control del comportamiento del modelo. <i>Ejemplo:</i> En <code>prompts.md</code>, cada versión del prompt del asistente registra un solo cambio (por ejemplo, exigir la frase exacta de derivación), su motivo y el efecto observado en las mismas preguntas de prueba. <i>Relacionados:</i> Zero-shot, Few-shot, Ajuste inicial, Protocolo de evaluación	Iteración y ajuste de prompts según resultados obtenidos.

Término	Definición	Contenido del plan
Max_tokens	Parámetro que fija el máximo de tokens de la respuesta: no hace que el modelo resuma, sino que corta la salida al llegar al límite, y entonces <code>choices[0].finish_reason</code> vale <code>length</code>. El largo se pide en el prompt y <code>max_tokens</code> queda como tope con margen que también acota el costo; algunos modelos recientes lo llaman <code>max_completion_tokens</code>. <i>Ejemplo:</i> Con un tope de 150 tokens, la respuesta del asistente, de una o dos oraciones en JSON, cabe con margen, y ninguna llamada puede gastar más de 150 tokens de salida. <i>Relacionados:</i> Token, Temperature, Salida estructurada	Prompt y parámetros: cómo influyen temperature, max_tokens y top_p.
Mensaje de sistema	Mensaje con rol <code>system</code> que abre la lista <code>messages</code> y fija el comportamiento general: el rol del asistente, sus reglas y el formato de salida. En la estructura básica de entrada le siguen los mensajes <code>user</code>, con la tarea y los datos de cada llamada, y los <code>assistant</code>, con respuestas previas o ejemplos. <i>Ejemplo:</i> «Eres el asistente de la mesa de ayuda de Nube Sur. Respondes en español, en tono cordial, solo con la respuesta publicada que se te entrega» es igual en todas las consultas; solo cambia el mensaje de usuario. <i>Relacionados:</i> Prompt, Restricción, Chat completions, Few-shot	Estructura básica de entrada para modelos de lenguaje. Prácticas recomendadas: claridad, delimitación del rol, contexto, restricciones y evitar ambigüedad.
Prompt	Entrada que recibe un modelo generativo para producir su respuesta; en un modelo de lenguaje es texto que dice qué tarea hacer y entrega los datos sobre los que hacerla. El modelo no lo ejecuta como un programa, sino que genera la continuación más probable: cada palabra influye y todo lo que debe saber tiene que estar escrito. <i>Ejemplo:</i> El prompt del asistente reúne las reglas, la pregunta frecuente elegida con su respuesta publicada y la consulta, porque el modelo no sabe nada más de Nube Sur. <i>Relacionados:</i> Mensaje de sistema, Zero-shot, Few-shot, Modelo de lenguaje	Qué es un prompt.
Prompt de validación	Prompt que revisa la salida de otro con una pregunta cerrada y fácil de procesar, por ejemplo si una respuesta contiene información que no está en su fuente. Es uno de los ejemplos prácticos de prompt, junto con los de generar código, resumen y reformulación. <i>Ejemplo:</i> Antes de mostrar una respuesta, el servidor de Nube Sur puede preguntar al modelo «¿Esta respuesta agrega algo que no esté en la respuesta publicada? Responde solo SI o NO». <i>Relacionados:</i> Alucinación, Restricción, Salida estructurada	Ejemplos prácticos: prompt para generar código, resumen, validación y reformulación.

Término	Definición	Contenido del plan
Restricción	Instrucción del prompt que fija lo que el modelo no puede hacer o el límite que debe respetar: no inventar datos, no proponer soluciones, un largo máximo, un idioma o una única fuente de información. Es una de las prácticas recomendadas, junto con la claridad, la delimitación del rol, el contexto y evitar la ambigüedad (por ejemplo, pedir «una o dos oraciones» en vez de «breve»). <i>Ejemplo:</i> «Responde solo con la información de la respuesta publicada; si no alcanza, escribe exactamente: Te derivo con un ejecutivo.» es la restricción que reduce el riesgo de que el asistente invente el precio del plan anual; la prueba con esa consulta verifica que la cumpla. <i>Relacionados:</i> Mensaje de sistema, Alucinación, Prompt de validación	Prácticas recomendadas: claridad, delimitación del rol, contexto, restricciones y evitar ambigüedad.
Salida estructurada	Respuesta del modelo con un formato definido de antemano (lista, JSON o tabla) para que la lea una persona o, sobre todo, otro programa; se pide indicando los campos, sus tipos y los valores permitidos. Se valida siempre, con <code>json.loads</code> dentro de un <code>try</code>, porque el modelo puede agregar texto o cortar la respuesta; algunos modelos aceptan además <code>response_format</code> para exigir JSON. <i>Ejemplo:</i> Para «Necesito pasar la lista de clientes a una planilla», el asistente devuelve <code>{"faq": "F3", "respuesta": "En Clientes, usa Exportar y elige el formato XLSX.", "derivar": false}</code>, y si el texto no es JSON válido el servidor lo registra y deriva. <i>Relacionados:</i> Parsing de respuesta, Instructional prompting, Max_tokens	Formateo de salidas: listas, JSON y estructura tabular.
Sensibilidad al wording	Limitación por la que un cambio pequeño en la redacción del prompt o de la pregunta, aunque diga lo mismo, cambia la respuesta del modelo. Por eso un prompt se prueba con varios casos y con paráfrasis de ellos, y se congela cuando funciona. <i>Ejemplo:</i> «¿funciona en safari?» recibe la respuesta de F4, pero «¿puedo usarla desde Safari?» podría recibir una respuesta más larga o una derivación; por eso cada pregunta de prueba se repite con dos o tres paráfrasis. <i>Relacionados:</i> Alucinación, Falta de control, Iteración de prompts, Restricción	Limitaciones: alucinaciones, falta de control y sensibilidad al wording.
Temperature	Parámetro de generación que controla la variedad de la respuesta: con valores bajos (0 a 0,3) el modelo elige casi siempre los tokens más probables y la salida varía poco entre corridas (ni siquiera con 0 se garantiza que sea idéntica); con valores altos aumentan la variedad y el riesgo de inventar. En la API de OpenAI va de 0 a 2, y algunos modelos de razonamiento no permiten cambiarla. <i>Ejemplo:</i> El asistente usa temperature 0,2 porque debe repetir con fidelidad una respuesta publicada, no crear una nueva. <i>Relacionados:</i> Top_p, Max_tokens, Alucinación	Prompt y parámetros: cómo influyen temperature, max_tokens y top_p.

Término	Definición	Contenido del plan
Top_p	Parámetro que limita la elección de cada token a los más probables cuya probabilidad acumulada llega a ese valor (muestreo de núcleo): con 0,1 solo entran las opciones de arriba y con 1 entran todas. La documentación de OpenAI recomienda ajustar temperature o top_p, no los dos a la vez. <i>Ejemplo:</i> Como el asistente ya controla la variedad con temperature 0,2, deja top_p en 1, su valor por defecto. <i>Relacionados:</i> Temperature, Token	Prompt y parámetros: cómo influyen temperature, max_tokens y top_p.
Zero-shot	Estrategia de prompting que pide la tarea solo con instrucciones, sin ejemplos resueltos. Es corta y barata en tokens, pero el modelo interpreta a su manera lo que no se dijo, como el largo, el tono o qué hacer en los casos especiales. <i>Ejemplo:</i> Con un prompt zero-shot, el asistente responde bien «¿funciona en safari?», pero ante una consulta que ninguna pregunta frecuente cubre a veces improvisa una respuesta en vez de derivar con «Te derivo con un ejecutivo». <i>Relacionados:</i> Few-shot, Instructional prompting, Iteración de prompts	Tipos de prompting: zero-shot, few-shot e instructional prompting.

Contenido 4 · Pipeline de preprocesamiento y evaluación básica de texto

Término	Definición	Contenido del plan
Ajuste inicial	Primer cambio concreto al prompt, a la limpieza, a los parámetros o a un umbral, decidido a partir de las métricas registradas: se comparan los promedios por configuración, se revisan los peores casos y se cambia una cosa a la vez, midiendo de nuevo con el mismo protocolo. <i>Ejemplo:</i> Si el registro muestra que con temperature 0,9 baja la relevancia y que la consulta por el precio del plan anual recibe una cifra inventada, el primer ajuste es bajar a 0,2 y volver a medir con el mismo protocolo; si esa consulta sigue recibiendo una cifra, el siguiente es reforzar la restricción de derivar, y se mide otra vez. <i>Relacionados:</i> Protocolo de evaluación, Iteración de prompts, Similitud coseno	Identificación de ajustes iniciales a partir de métricas obtenidas.
BLEU	Métrica que mide cuánto del texto generado aparece en la referencia, contando coincidencias de n-gramas de hasta cuatro tokens, y penaliza los textos demasiado cortos; va de 0 a 1. En textos cortos se suaviza (en NLTK, con <code>SmoothingFunction</code>), porque sin suavizado da 0 con facilidad. <i>Ejemplo:</i> Si la respuesta a «Necesito pasar la lista de clientes a una planilla» no comparte ninguna secuencia de cuatro palabras con su referencia, BLEU sin suavizado da 0 aunque la respuesta sea correcta. <i>Relacionados:</i> ROUGE, N-grama, Protocolo de evaluación	Métricas básicas de calidad de respuestas: coherencia, relevancia, BLEU y ROUGE.

Término	Definición	Contenido del plan
Coherencia	Métrica de calidad que valora una persona con una pauta (en este módulo, de 1 a 3) e indica si la respuesta se entiende por sí sola: vale 3 si es gramatical, ordenada y sin contradicciones. <i>Ejemplo:</i> Una respuesta que dice que el plan se cambia en Facturación → Plan y en la oración siguiente que no se puede cambiar recibe coherencia 1, aunque comparta muchas palabras con la respuesta publicada de F5. <i>Relacionados:</i> Relevancia, ROUGE, Protocolo de evaluación	Métricas básicas de calidad de respuestas: coherencia, relevancia, BLEU y ROUGE.
CountVectorizer	Clase de scikit-learn que arma un vocabulario con las palabras de un conjunto de textos y representa cada texto como un vector con la cantidad de veces que aparece cada una (bolsa de palabras), sin distinguir una palabra rara de una que está en todos los textos. Su parámetro <code>stop_words</code> solo trae incorporada la lista en inglés, así que para español se le entrega una lista, por ejemplo la de NLTK. <i>Ejemplo:</i> Con CountVectorizer sobre las ocho preguntas frecuentes, «cómo», que aparece en la mitad de ellas, cuenta lo mismo que «contraseña», que solo está en F1 y es la palabra que la distingue. <i>Relacionados:</i> TF-IDF, Stopwords, Similitud coseno	Vectorización básica: CountVectorizer y TF-IDF.
Lema	Forma de diccionario de una palabra, como «poder» para «pude» o «ticket» para «tickets». La lematización lleva cada palabra a su lema con el análisis gramatical de un modelo del idioma (<code>token.lemma_</code> en spaCy): da palabras reales, pero es más lenta que el stemming y el modelo pequeño a veces se equivoca. <i>Ejemplo:</i> Para contar los temas más frecuentes de las consultas del mes, Nube Sur lematiza, y así «facturas» y «factura» suman juntas y el informe muestra palabras reales. <i>Relacionados:</i> Stemming, Tokenización, Token	Conceptos básicos de NLP: token, palabra, lema, stopwords y n-gramas. Lematización y stemming.
Limpieza de texto	Quitar del texto todo lo que no es el mensaje: etiquetas y entidades HTML, caracteres especiales repetidos y ruido como firmas, «Enviado desde mi teléfono» o enlaces, reemplazando los datos personales por marcas como <code>[correo]</code> o <code>[enlace]</code>. Se hace con <code>html.unescape</code> y expresiones regulares del módulo <code>re</code>, cuidando de no borrar lo que es parte del problema. <i>Ejemplo:</i> La función <code>limpiar</code> deja el ticket T1 sin etiquetas <code><p></code> ni la firma con el correo de Marta, pero conserva los caracteres rotos «Compañía» del ticket T2, porque son la falla que se reporta. <i>Relacionados:</i> Normalización de texto, Pipeline de preprocesamiento, Token	Técnicas de limpieza: eliminación de caracteres especiales, html y ruido.

Término	Definición	Contenido del plan
N-grama	Secuencia de n tokens consecutivos: unigrama con uno, bigrama con dos, trigramas con tres. Captura expresiones que una palabra sola no dice, y lo usan tanto la vectorización (<code>ngram_range</code> en scikit-learn) como las métricas BLEU y ROUGE. <i>Ejemplo:</i> El bigrama «iniciar sesión» identifica el problema del ticket T1 mejor que «iniciar» o «sesión» por separado. <i>Relacionados:</i> Token, TF-IDF, BLEU	Conceptos básicos de NLP: token, palabra, lema, stopwords y n-gramas.
Normalización de texto	Hacer que textos equivalentes se escriban igual: pasar a minúsculas, unificar espacios y saltos de línea y estandarizar caracteres, comillas, fechas o montos. Lo que va al modelo se deja natural, con mayúsculas y tildes; la normalización más agresiva, como quitar tildes, se aplica solo para comparar y medir. <i>Ejemplo:</i> Al medir, «Sesion» y «sesión» pasan a «sesion» con <code>lower()</code> y <code>quitar_tildes</code>, y cuentan como la misma palabra en ROUGE. <i>Relacionados:</i> Limpieza de texto, Tokenización, ROUGE	Normalización de mayúsculas, minúsculas y estandarización de texto.
Pipeline de preprocesamiento	Secuencia de etapas por la que pasa cada texto antes de llegar al modelo o a la medición (limpiar, normalizar, tokenizar, vectorizar), con estructura modular: cada etapa es una función reutilizable con una sola responsabilidad, que se prueba, se cambia y se reutiliza por separado. <i>Ejemplo:</i> <code>preprocesar.py</code> reúne <code>limpiar</code>, <code>quitar_tildes</code>, <code>tokenizar</code> y <code>ngramas</code>: <code>limpiar</code> se aplica a las preguntas frecuentes y a las consultas antes de pedir sus embeddings, y <code>quitar_tildes</code> y un mismo tokenizador se aplican a las respuestas generadas y a las de referencia antes de medir, para que se comparen igual. <i>Relacionados:</i> Pipeline, Limpieza de texto, Tokenización, Modularización	Estructura modular de un pipeline de preprocesamiento. Funciones reutilizables para cada etapa.
Protocolo de evaluación	Conjunto de reglas fijas con que se evalúa para que los resultados sean comparables: qué casos se usan, contra qué referencias, con qué tokenizador y métricas, qué pauta aplican las personas, qué umbral marca un caso para revisar y cómo se registra cada prueba. El registro lleva una fila por caso y configuración en un CSV, incluidas las pruebas que salen mal. <i>Ejemplo:</i> El <code>resultados.csv</code> del asistente tiene una fila por pregunta de prueba y versión de prompt, con la pregunta frecuente elegida, ROUGE-L, BLEU, coherencia, relevancia y un comentario, y los tokens de cada llamada van aparte en <code>consumo.csv</code>. <i>Relacionados:</i> ROUGE, BLEU, Ajuste inicial, Iteración de prompts	Registro de resultados de evaluación.

Término	Definición	Contenido del plan
Relevancia	Métrica de calidad, valorada por una persona con una pauta de 1 a 3, que indica si la respuesta atiende exactamente lo pedido sin agregar nada que no esté en la fuente: en el resumidor vale 3 si nombra la falla y el producto del ticket, y en el asistente, si responde con la respuesta publicada que corresponde. Puede aproximarse con la similitud coseno entre la entrada y la respuesta, pero la valoración humana detecta lo que las métricas de coincidencia no ven. <i>Ejemplo:</i> Si a «quiero sumar a mi colega como usuario» el asistente responde cómo descargar facturas (F6), la respuesta puede ser clara y gramatical, pero su relevancia es 1. <i>Relacionados:</i> Coherencia, Similitud coseno, Alucinación	Métricas básicas de calidad de respuestas: coherencia, relevancia, BLEU y ROUGE.
ROUGE	Familia de métricas que mide cuánto de un texto de referencia aparece en el texto generado: ROUGE-1 compara palabras sueltas y ROUGE-L, la subsecuencia común más larga, que respeta el orden. Se suele reportar su F1, de 0 a 1, y mide coincidencia de palabras, no verdad. <i>Ejemplo:</i> Si ROUGE-L sale bajo en todas las respuestas en español, lo primero es revisar el tokenizador: el de <code>rouge-score</code> por defecto descarta las letras con tilde y la ñ, y por eso <code>evaluar.py</code> usa <code>TokenizadorES</code>. <i>Relacionados:</i> BLEU, Coherencia, Relevancia, Protocolo de evaluación	Métricas básicas de calidad de respuestas: coherencia, relevancia, BLEU y ROUGE.
Similitud coseno	Medida de cuánto se parecen dos vectores según el ángulo entre ellos: 1 es la misma dirección y 0, ninguna relación; con TF-IDF va de 0 a 1 y con embeddings puede ser negativa. Se calcula, por ejemplo, con <code>cosine_similarity</code> de scikit-learn, sobre vectores TF-IDF o embeddings. <i>Ejemplo:</i> El asistente elige la pregunta frecuente con mayor similitud coseno a la consulta y, si ese valor queda bajo su umbral de derivación, deriva sin llamar al modelo de generación, lo que ahorra tokens y evita inventar. <i>Relacionados:</i> Embedding, TF-IDF, Vector store, Ajuste inicial	Vectorización básica: CountVectorizer y TF-IDF.
Stemming	Técnica que reduce una palabra a su raíz recortando sufijos con reglas fijas, sin analizarla, como hace <code>SnowballStemmer("spanish")</code> de NLTK. Es muy rápida y no necesita modelo, pero la raíz puede no ser una palabra. <i>Ejemplo:</i> «facturación» y «facturar» quedan en «factur», lo que sirve para buscar rápido en miles de tickets antiguos, pero no para mostrar en un informe. <i>Relacionados:</i> Lema, Tokenización	Lematización y stemming.

Término	Definición	Contenido del plan
Stopwords	Palabras muy frecuentes y de poco contenido propio, como artículos, preposiciones y conjunciones («el», «de», «que», «y»), que se quitan cuando interesa el tema de un texto y no su redacción. Las listas en español, como la de NLTK, incluyen «no», que en un ticket puede ser justo el problema. <i>Ejemplo:</i> Sin stopwords, de «No puedo iniciar sesión en la app» quedan palabras con contenido como iniciar, sesión y app, pero se pierde la negación, que un resumen sí debe conservar. <i>Relacionados:</i> Token, CountVectorizer, Tokenización	Conceptos básicos de NLP: token, palabra, lema, stopwords y n-gramas.
TF-IDF	Forma de vectorizar que pondera cada palabra por su frecuencia en el texto (TF) y por su rareza en el conjunto (IDF), de modo que pesan más las palabras que distinguen a un texto. En scikit-learn se usa <code>TfidfVectorizer</code>, que admite n-gramas con <code>gram_range</code>. <i>Ejemplo:</i> La búsqueda base del asistente vectoriza con TF-IDF las ocho preguntas frecuentes y la consulta, y muestra su límite frente a los embeddings: «olvidé la clave y no puedo entrar» no comparte ninguna palabra con F1, así que su similitud con ella es 0. <i>Relacionados:</i> CountVectorizer, Similitud coseno, Embedding, N-grama	Vectorización básica: CountVectorizer y TF-IDF.
Token	Unidad en que se divide un texto para procesarlo: para spaCy o NLTK suele ser una palabra o un signo de puntuación, y para un modelo de lenguaje es una pieza de su vocabulario (una palabra, parte de una palabra o un signo), así que no siempre coincide con una palabra. En los modelos por API es también la unidad en que se miden los límites y se cobra el consumo. <i>Ejemplo:</i> Quitar el HTML y la firma del ticket T1 antes de enviarlo reduce los <code>prompt_tokens</code> que informa <code>usage</code> y, con ellos, el costo de la llamada. <i>Relacionados:</i> Tokenización, Max_tokens, N-grama, Parsing de respuesta	Conceptos básicos de NLP: token, palabra, lema, stopwords y n-gramas.
Tokenización	Dividir un texto en tokens con las reglas del idioma, no solo por espacios, para separar bien signos como «¿» o una coma pegada a la palabra. spaCy lo hace con un modelo del español (<code>es_core_news_sm</code>) que además indica el lema de cada token y si es stopword o puntuación; NLTK ofrece <code>word_tokenize</code> y listas de stopwords, cuyos recursos se descargan la primera vez con <code>nlk.download</code>. <i>Ejemplo:</i> <code>tokenizar(limpio, sin_stopwords=True)</code>, de <code>preprocesar.py</code>, deja del ticket T1 limpio, entre otros, <code>iniciar</code>, <code>sesión</code>, <code>app</code>, <code>facturación</code> y <code>contraseña</code>, sin artículos ni puntuación; con <code>split()</code>, «contraseña» habría quedado con el punto pegado. <i>Relacionados:</i> Token, Lema, Stopwords, Normalización de texto	Tokenización y normalización con librerías como spaCy o NLTK.

Índice alfabético

Término	Contenido	Término	Contenido
Ajuste inicial	4	Normalización de texto	4
Alucinación	3	Parsing de respuesta	2
API REST	2	Pipeline	1
API simulada	2	Pipeline de preprocesamiento	4
Bearer	2	Prompt	3
BLEU	4	Prompt de validación	3
Chat completions	2	Protocolo de evaluación	4
Código de estado	2	Prueba unitaria	2
Coherencia	4	QA (preguntas y respuestas)	1
CountVectorizer	4	Referencia de la API	2
Docstring	2	Relevancia	4
Embedding	1	Restricción	3
Endpoint	2	ROUGE	4
Falta de control	3	Salida estructurada	3
Few-shot	3	Sensibilidad al wording	3
httpx	2	Servidor API	1
Hugging Face Transformers	1	Similitud coseno	4
IA basada en reglas	1	Solicitud–respuesta	1
IA generativa	1	Stemming	4
Instructional prompting	3	Stopwords	4
Iteración de prompts	3	Summarization	1
LangChain	1	Temperature	3
Lema	4	TF-IDF	4
Limpieza de texto	4	Token	4
Max_tokens	3	Tokenización	4
Mensaje de sistema	3	Top_p	3
Modelo autorregresivo	1	Transformer	1
Modelo de difusión	1	Variable de entorno	2

Término	Contenido	Término	Contenido
Modelo de lenguaje	1	Vector store	1
Modularización	2	Zero-shot	3
N-grama	4		